L'intelligence artificielle au service de la détection des vulnérabilités logicielles : état de l'art et étude de cas VulHGT

Sèdofi Hope LANMAYIKPOHOUE[(1*)], Mahugnon Géraud AZEHOUN PAZOU[(1)]

[(1)] Université Nationale des Sciences, Technologies, Ingénierie et Mathématiques (UNSTIM), Abomey, Bénin

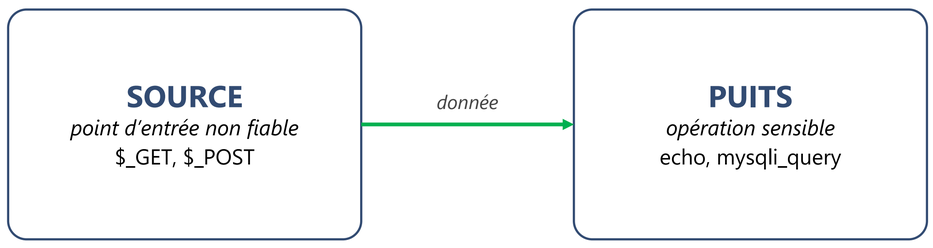
CSI-UNSTIM
3e Edition

CONTEXTE ET OBJECTIF

Contexte. La transformation numérique en Afrique de l'Ouest s'appuie largement sur des applications développées en PHP, qui alimente environ 74 % du web. Ces systèmes restent exposés à des vulnérabilités critiques comme le Cross-Site Scripting (XSS) et l'injection SQL (SQLi), alors que les outils d'analyse statique traditionnels souffrent d'un taux élevé de faux positifs et d'un rappel limité, tandis que les approches d'apprentissage profond fondées sur des représentations séquentielles ou des graphes homogènes peinent à capturer les dépendances multiples du code.

Objectif. Ce travail présente VulHGT, un framework qui combine les graphes de propriétés de code (Code Property Graphs) avec des Heterogeneous Graph Transformers (HGT), conçu pour détecter les vulnérabilités XSS (CWE-79) et SQLi (CWE-89) dans le code source PHP.

Le principe source-puits



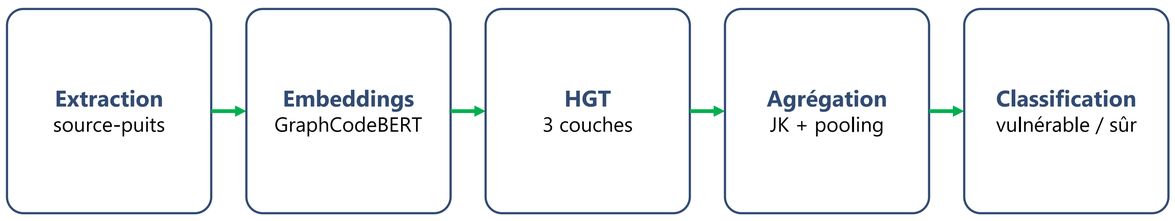
Vulnérabilité : la donnée atteint le puits sans validation ni nettoyage intermédiaire.

MÉTHODOLOGIE

ARCHITECTURE

Une extraction adaptative de sous-graphes source vers puits capture, pour chaque fonction, les dépendances syntaxiques, de flux de contrôle et de flux de données pertinentes.

- Nœuds initialisés par des embeddings GraphCodeBERT.
- Trois couches de HGT à attention typée selon la relation.
- Agrégation par Jumping Knowledge.
- Pooling multi-vue (moyenne, somme, attention) avant classification.



Pipeline VulHGT, du code PHP à la décision

Les quatre relations exploitées par l'attention typée

Relation	Ce qu'elle décrit
AST	La structure syntaxique du code
CFG	L'ordre d'exécution des instructions
DDG	La circulation des données entre instructions
CDG	La dépendance d'une instruction à une condition

ÉVALUATION

Jeu de données SARD (NIST)

Vulnérabilité	Fichiers PHP	Part vulnérable
XSS (CWE-79)	159 364	17,9 %
SQLi (CWE-89)	94 527	21,3 %
Total	253 891	

Répartition stratifiée



Le jeu de test reste isolé jusqu'à l'évaluation finale.

Métriques. Précision, rappel, F1-score et AUC-PR, privilégiés à l'exactitude brute en raison du déséquilibre des classes.

Comparaison contrôlée. Tous les modèles partagent les mêmes embeddings GraphCodeBERT.

Modèle	Graphe reçu	Rôle
GCN, GATv2, GGNN	Structure syntaxique seule	Références homogènes
GGNN (graphe complet)	Identique à celui de VulHGT	Isoler l'attention hétérogène
VulHGT	Complet, relations typées	Modèle proposé

RÉSULTATS ET CONCLUSION

99,44 % F1-score XSS (SARD)

91,44 % F1-score SQLi (SARD)

100 % Rappel XSS (CloudClassroom)

94,1 % Rappel SQLi (CloudClassroom)

Résultats. VulHGT devance les outils d'analyse statique (RIPS, WAP, Progpilot) et les architectures homogènes, y compris le GGNN recevant le même graphe complet (+0,66 point de F1 en XSS, +1,95 en SQLi) : un gain modeste, mais propre à l'attention hétérogène. Sur CloudClassroom, application PHP réelle de 57 fichiers non vue à l'entraînement, le rappel reste élevé mais la précision baisse (64,9 % en XSS, 58,2 % en SQLi) : les échantillons synthétiques de SARD ne couvrent pas toute la diversité du code réel. L'analyse complète de ce projet prend moins de 45 secondes, ce qui rend VulHGT intégrable dans une chaîne DevSecOps.

Comparaison sur les mêmes jeux de test (rappel)

Jeu de test	VulEye	Dtector	VulHGT
XSS-Labs (18 fichiers)	72,2 %	88,9 %	100 %
SQLi-Labs (69 fichiers)	88,4 %	89,9 %	91,3 %

Jeux entièrement vulnérables : seul le rappel y est mesurable. VulEye et Dtector sont deux approches PHP fondées sur des graphes homogènes.

Conclusion. Ces résultats confirment l'intérêt d'une modélisation hétérogène des graphes de propriétés de code pour la détection de vulnérabilités PHP, tout en soulignant les limites des données synthétiques d'entraînement pour la généralisation en contexte réel. Ce travail plaide pour un investissement dans la recherche en cybersécurité augmentée par l'IA au sein de l'écosystème scientifique béninois, en cohérence avec les ambitions nationales de transformation numérique.

Références

Hu Z., Dong Y., Wang K., Sun Y. Heterogeneous graph transformer. The Web Conference, 2020.
Lin C., Xu Y., Fang Y., Liu Z. VulEye: a novel graph neural network vulnerability detection approach for PHP. Applied Sciences, 2023.
Nguyen T.N., Vo N.S., Le D.T., Nguyen-An K.. A study on deep graph neural networks for security vulnerabilities detection in web applications. FDSE, 2024.
Lanmayikpohoue S.H., Assogba E.K., Azehoun Pazou M.G., Aoga J.O.R., Konnon M.A. VulHGT: leveraging code property graphs and heterogeneous graph transformers for PHP vulnerability detection. CARI 2026 (accepté).

Remerciements

Ressources de calcul fournies par l'IMSP (Dangbo), maintenues par Calcul Bénin.

E-mails : slanmayikpohoue@unstim.bj ; geraud.pazou@unstim.bj